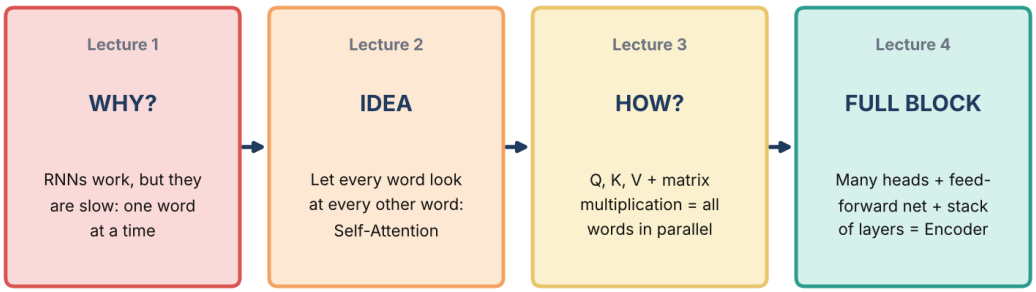
Week 1 · Lectures 1–4

# Transformers

## From “RNNs are slow” to the full Transformer encoder




*The story of Week 1: from a problem, to an idea, to the maths, to the full building block*

*The whole week in one picture. Each lecture answers the question the previous one left open.*

> **★ The big picture in 30 seconds**
>
> An **RNN** reads a sentence **one word at a time**: word 2 must wait for word 1, word 3 must wait for word 2, and so on. That is slow, even though the whole sentence was available from the start. A **Transformer** lets **every word look at every other word at the same moment**. This trick is called **self-attention**, and it turns the whole job into a few big matrix multiplications that can run **in parallel**. This week you learn exactly how that works, step by step.

### How to read these notes

- Every lecture starts with a **big question**, then answers it with pictures and one small example.
- Boxes: **green** = key idea · **orange** = analogy / example · **red** = problem or warning · grey = self-check.
- Colours in diagrams: **Query (q)** = orange, **Key (k)** = teal, **Value (v)** = purple. Red = slow/sequential, green = fast/parallel.
- Numbers in the heat-maps and the tiny worked examples are **made up for illustration**. The ideas and formulas come from the lectures.

### Contents


| Part | Topic | You will be able to… |
|---|---|---|
| **1** | Introduction to Transformer architecture | explain why RNNs are slow and what attention already fixed |
| **2** | “Attention is all you need”: the architecture explained | explain self-attention and why the word “it” needs it |
| **3** | Self-attention: contextual word embeddings | compute q, k, v, scores and z by hand, and as matrices |
| **4** | Multi-headed attention | explain heads, feed-forward layer and the whole encoder |
| **End** | Cheat-sheet, common confusions, answers | revise everything on 2 pages |

## Before you start: words and symbols

| Term | Plain meaning |
|---|---|
| **Token / word** | One piece of the input sentence. T = how many tokens the sentence has. |
| **Embedding** | A list of numbers (a vector) that stands for a word. Example: 512 numbers per word. |
| **Contextual representation** | A vector for a word **as it is used in this sentence**, so it knows about the words around it. |
| **Sequential vs parallel** | Sequential = one after another (must wait). Parallel = many at the same time. |
| **Attention weight (α)** | A number between 0 and 1 saying how much one word should listen to another. All weights for one word add up to 1. |
| **Softmax** | Turns any list of numbers into weights that are positive and add up to 1. |
| **Encoder / Decoder** | Encoder reads the input sentence. Decoder writes the output (e.g., the translation) one word at a time. |

**⚠ Heads-up: the lectures change the letters!**

The lecturer re-uses letters while moving from RNNs to Transformers, which is confusing on first read. Use this table as your decoder ring.

| Symbol | In Lecture 1 (RNN + attention) | In Lectures 2–4 (Transformer) |
|---|---|---|
| $x_i$ | input word embedding | input word embedding (same idea) |
| $h_i$ | output of the RNN encoder (contextual vector) | **input vectors** to the self-attention layer (the word embeddings) |
| $s_t$ | decoder state at time t | briefly: the **new vector** built by self-attention (L2, L4); then passed to the feed-forward net |
| $z_i$ | – | output of self-attention (L2–L3); final output of the layer (L4) |
| $c_t$ | context vector (weighted sum of h) | – |
| $E_1 \ldots E_T$ | – | final output of the whole encoder (after all layers) |
| **α, e** | attention weight, raw score | attention weight, raw score (same idea) |

**✔ Rule of thumb**

Ignore the letters and track the **idea**: *a list of T vectors goes in, a better list of T vectors comes out.* In an RNN the better vectors are built one by one. In a Transformer they are built all together.

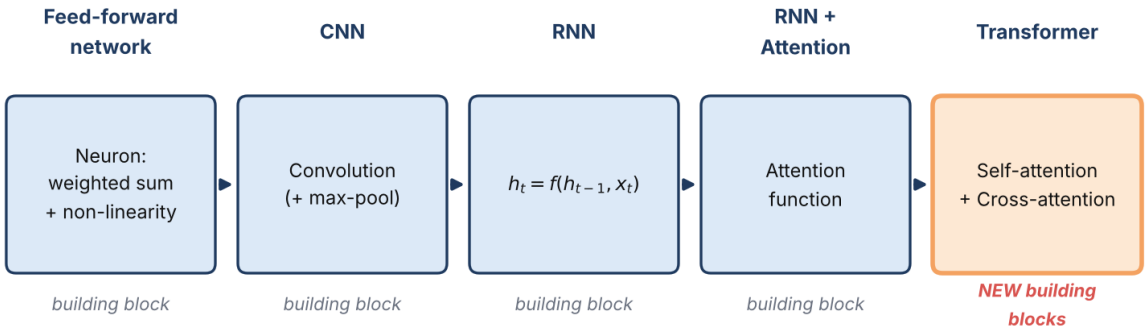
LECTURE 1

# Introduction to Transformer Architecture

*Big question: RNNs are good at understanding sentences. So why do we want to replace them?*

## 1.1 Where are we? A recap of building blocks

Every network you have met becomes easy once you know its **one basic building block**. Stack many blocks and you get a deep, wide network. Today we meet the building blocks of the Transformer (**self-attention** and **cross-attention**) and relate them to the attention you already saw with RNNs.

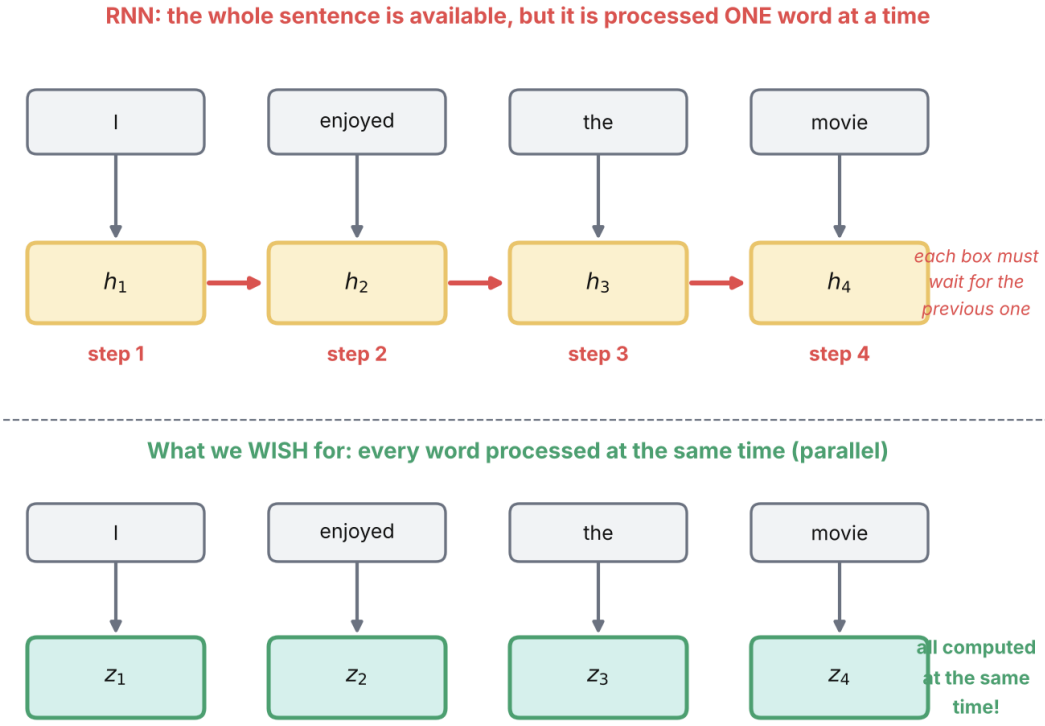



*Basic building blocks of each architecture so far. The Transformer introduces two new ones.*

## 1.2 The problem: an RNN computes one step at a time

Task: translate “I am going home”. Notice two things:

- The **input is given all at once**. We do not receive it one word at a time.
- But the RNN **computes it one word at a time**: $h_1 = f(h_0, x_1)$, then $h_2 = f(h_1, x_2)$, then $h_3$... You cannot compute $h_2$ until $h_1$ is finished.




*Top: the RNN waits at every step. Bottom: what we would like, with all words processed at the same time.*

✦ **Analogy: a relay race**

All the runners are already standing on the track (the whole sentence is available), but runner 2 cannot start until runner 1 hands over the baton. We would like all runners to run at once.

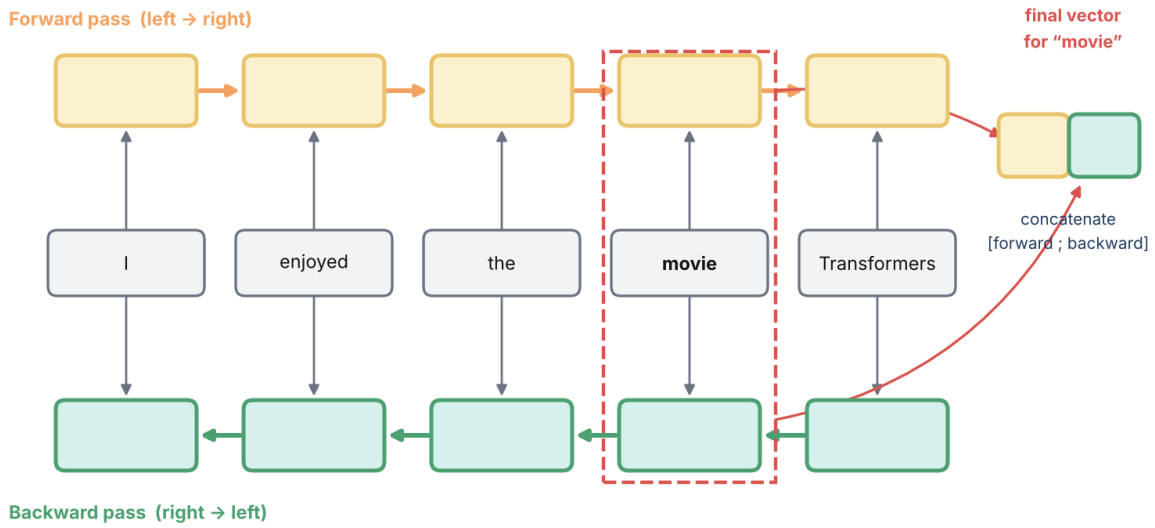
## 1.3 Why does the RNN do this? To get contextual representations

A word embedding (from Word2Vec, GloVe or FastText, or just learned from scratch) is the **same vector everywhere**. But the meaning of a word depends on its sentence. The RNN's yellow boxes fix this: each $h_t$ takes input from the words before it, so it becomes **aware of its context**.

✦ **Example: the word "bank"**

"I sat on the river **bank**" vs. "I deposited cash in the **bank**". A plain embedding gives "bank" the **same** vector in both sentences. A contextual representation gives **different** vectors, because the neighbouring words (river / cash) change what "bank" means.

One problem: reading only left to right means "movie" never sees "Transformers" (the word after it). The fix is a **bidirectional RNN**: run one RNN forward, another backward, then **concatenate** the two vectors for each word.




*Bidirectional RNN. The final vector for "movie" = [forward vector ; backward vector] joined side by side.*

⚠ **Good thing vs. bad thing**

**Good:** contextual representations (every word knows its neighbours). We want to keep this.
**Bad:** to get them, we process words one by one, which kills parallelism. We want to get rid of this.

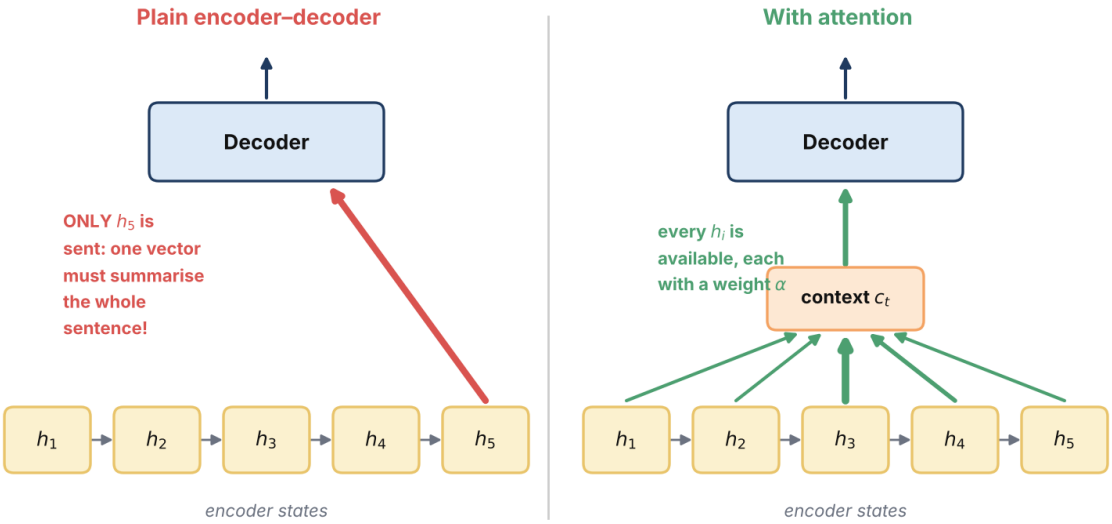
## 1.4 The decoder is sequential, and that is OK

The decoder produces the translation one word at a time, and each new word is fed back as the next input. So it **must** be sequential; there is no way around that. The question for this week is about the **encoder**, where the whole input is known in advance: *can we speed it up?*

## 1.5 From a bottleneck to attention

In the basic encoder–decoder, only the **last state** ($h_5$) is passed to the decoder. That single vector must summarise the entire sentence, and there is no notion of **alignment**: when producing the first

output word, the decoder cannot focus on the input word “I”. **Attention** fixes exactly this.

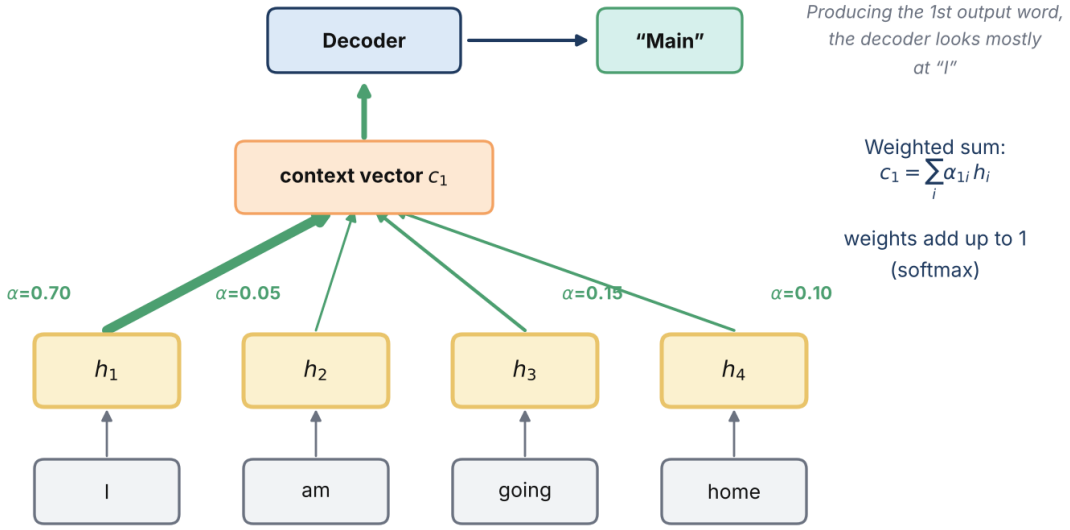



*Left: one vector carries everything (bottleneck). Right: the decoder can look at all encoder states, each with its own weight.*

How attention works (from the RNN lectures):

1. Run the encoder and keep all the vectors $h_1 \ldots h_T$. We no longer need the RNN block itself, just these T vectors.
2. At each output step t, compute a **weight $\alpha_{ti}$** for every input word. A softmax makes the weights positive and sum to 1.
3. Take the **weighted sum** of the h vectors. This is the **context vector** $c_t$ (also called the “thought vector”), and it is fed to the decoder.
4. Repeat for the next output word with new weights.

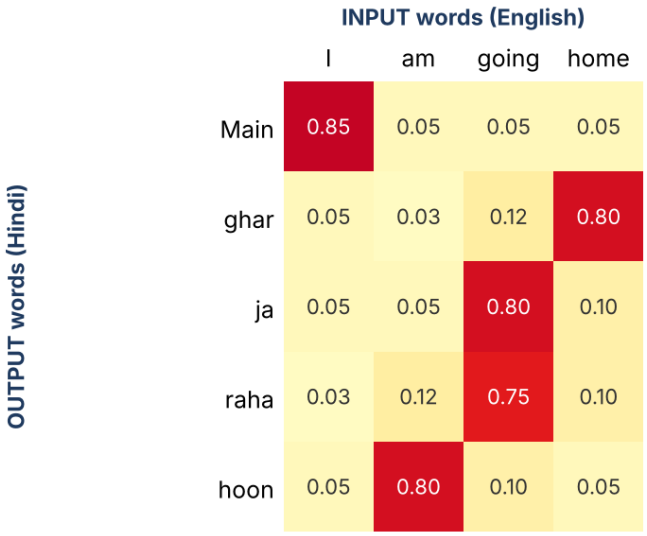
$$c_t = \sum_{i=1}^{T} \alpha_{ti} h_i \qquad \text{where} \quad e_{ti} = f(s_{t-1}, h_i) \;\Rightarrow\; \alpha_{ti} = \text{softmax}(e_{ti})$$




*Building the context vector $c_1$ while producing the first output word. Thicker arrow = bigger weight.*

**Matrix view:** put $h_1 \ldots h_5$ as columns of a matrix and multiply by the vector of weights. This is just a linear combination of the columns: $\alpha_{11}$·(column 1) + $\alpha_{12}$·(column 2) + …

**Heat-map:** plotting all the weights gives a grid (output words × input words). A bright spot means "while producing this output word, the model paid most attention to that input word". In translation these spots often form a near-diagonal line because words roughly correspond one-to-one.



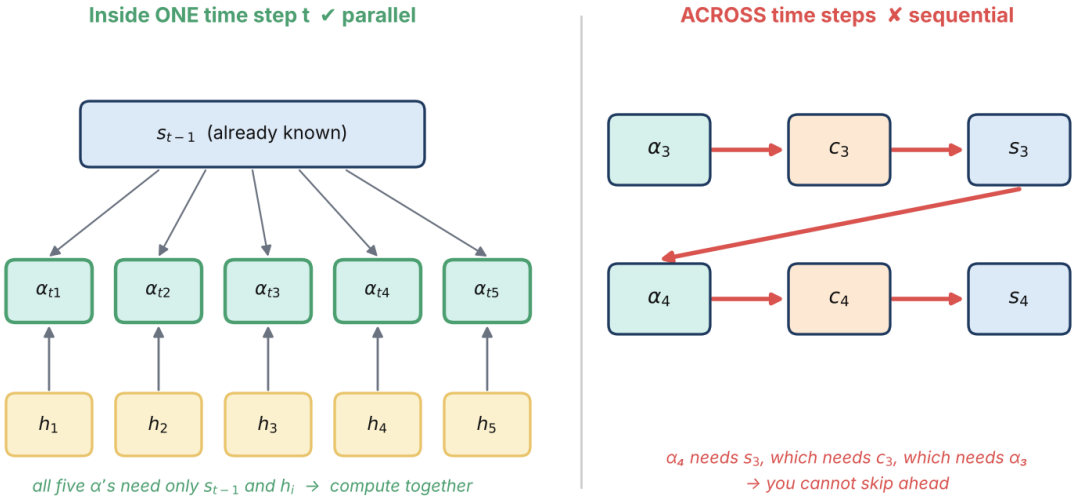

*darker = more attention (illustrative numbers)*

*A toy attention heat-map for "I am going home" → "Main ghar ja raha hoon". Numbers are illustrative.*

## 1.6 Can attention be parallelised? (the key question)

The weight is computed by a scoring function of the previous decoder state and an input vector: $e_{ti} = f(s_{t-1}, h_i)$, followed by softmax. Two questions:

| Question | Answer | Why |
|---|---|---|
| **At one fixed time step t, can we compute all $\alpha_{t1}…\alpha_{tT}$ together?** | **YES** | Each needs only $s_{t-1}$ and $h_i$, and both are already known. (Softmax needs all scores, but the scores themselves are independent.) |
| **Can we compute the α's for all time steps t together?** | **NO** | $\alpha_t$ needs $s_{t-1}$ → which needs $c_{t-1}$ → which needs $\alpha_{t-1}$. Each step waits for the previous one. |




*Left: inside one time step everything is independent. Right: across time steps there is a chain of dependencies.*

**✓ The big idea that leads to Transformers**

Attention itself **can** be parallelised. The only thing blocking us is the **recurrent connection** ($s_{t-1}$ feeding the next step). If we remove the recurrence and keep attention, maybe we can compute all the α's at once. That is the Transformer.

## 1.7 Summary of Lecture 1

- RNNs work very well (translation, summarisation, image/video captioning), but each step must wait for the previous one.
- We like **contextual representations** and **attention**. We dislike the **sequential computation** that comes from recurrence.
- **Wish list:** keep attention ✔ · keep contextual vectors ✔ · remove recurrence → get parallelism ✔

**✎ Self-check (answers at the end of the notes)**

**1.** Why does an RNN compute $h_2$ only after $h_1$, even though the whole sentence is available?

**2.** What is the difference between a plain word embedding and a contextual representation? Give your own example.

**3.** In attention, why can the α's of one time step be computed in parallel, but not the α's of all time steps?

LECTURE 2

# “Attention Is All You Need”: the Transformer explained

*Big question: How can we build contextual representations WITHOUT a recurrent connection?*

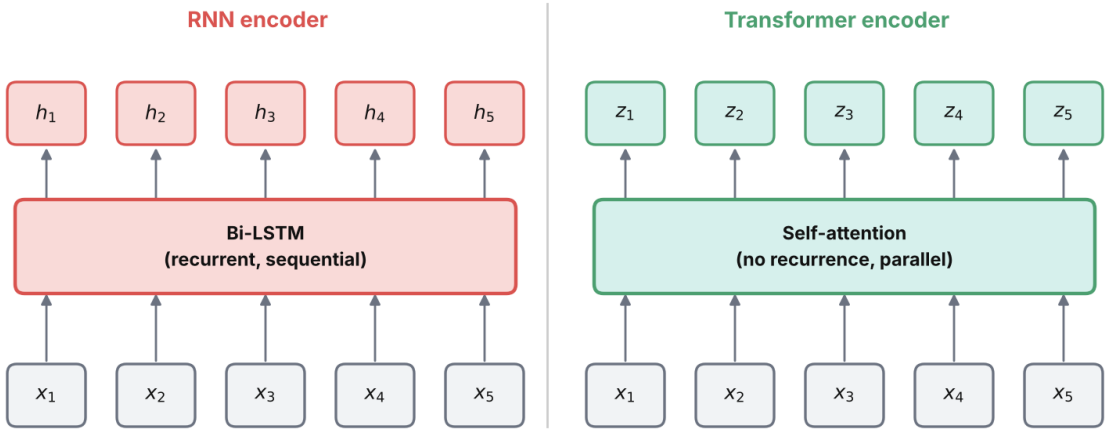
## 2.1 From RNN to Transformer: the 30,000-foot view

We have two earlier models: (a) the plain encoder–decoder RNN, which has recurrence, and (b) the attention-based version, where attention is parallel within one step but still waits across steps. The **Transformer** is also an encoder–decoder, but it is built from different blocks:

- **Encoder:** self-attention → feed-forward network
- **Decoder:** self-attention → encoder–decoder attention (cross-attention) → feed-forward network
- So there are only **three ideas** to learn: self-attention, encoder–decoder attention, feed-forward network. This week focuses on the **encoder side**: self-attention + feed-forward.

**What stays the same as the RNN model?**

- The input sentence is still given to us in full, and the output is still produced one word at a time.
- We still feed in **word embeddings** (in practice usually randomly initialised and learned, not copied from Word2Vec).
- The encoder still turns T input vectors into T **contextual** output vectors.




*Same job, different machinery. Both encoders turn word vectors x into contextual vectors (called h for the RNN, z for the Transformer).*

| | RNN encoder | Transformer encoder |
|---|---|---|
| **Input** | $x_1 \ldots x_T$ (word embeddings) | $x_1 \ldots x_T$ (word embeddings) |
| **Output** | $h_1 \ldots h_T$ (contextual) | $z_1 \ldots z_T$ (contextual) |
| **How context is gathered** | $h_t$ depends on $h_{t-1}$ (recurrence) | each word looks at all words directly (attention) |
| **Computation** | sequential | parallel |

## 2.2 Self-attention in one sentence

**✔ Key idea**

To build the **new vector for one word**, take a **weighted sum of the vectors of ALL the words in the sentence** (including the word itself). The weights say how relevant each word is to the current word.

$$s_i = \sum_{j=1}^{T} \alpha_{ij} h_j \qquad \text{where} \quad \alpha_{ij} = \text{softmax}_j(f(h_i, h_j))$$

**Reading the formula:** to compute the new vector $s_4$ for word 4, compute weights $\alpha_{41} \ldots \alpha_{4T}$ from pairs $(h_4, h_j)$ and add up $\alpha_{4j} \cdot h_j$. Do the same for word 2, word 5, and so on. *(In this lecture the new vector is temporarily called s, and the inputs h. Later the output is renamed z.)*

**Why can this be parallel?** The old attention needed $s_{t-1}$, which only appears after the previous step finishes. Here the scoring function uses only $h_i$ and $h_j$, and **we already have every h** (we just looked up the embeddings). Nothing waits for anything.

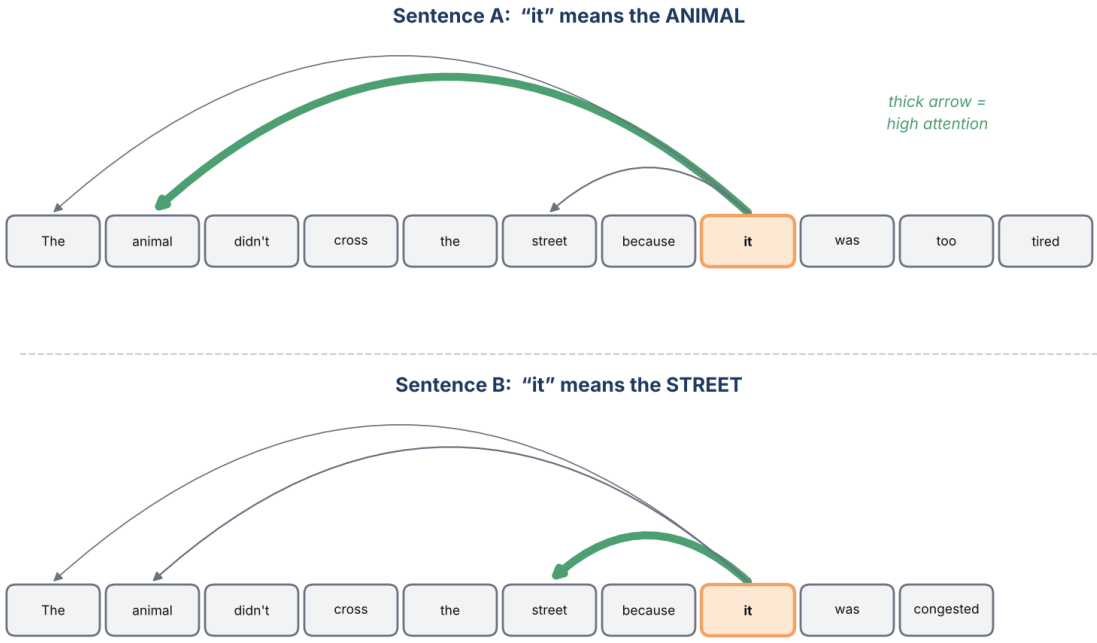
**✦ Self-attention vs. cross-attention**

**Cross-attention** (Lecture 1): attention *between* the decoder and the encoder words. "While writing the output, which input words matter?"

**Self-attention** (today): attention *within* the same sentence. "To understand this word, which other words of the same sentence matter?"

## 2.3 Why do we need it? The word "it"

Take: *"The animal didn't cross the street because it was too tired."* Here "it" refers to **animal**. Change the end to *"...because it was congested"* and now "it" refers to the **street**. The word is identical, but the right context word changes.



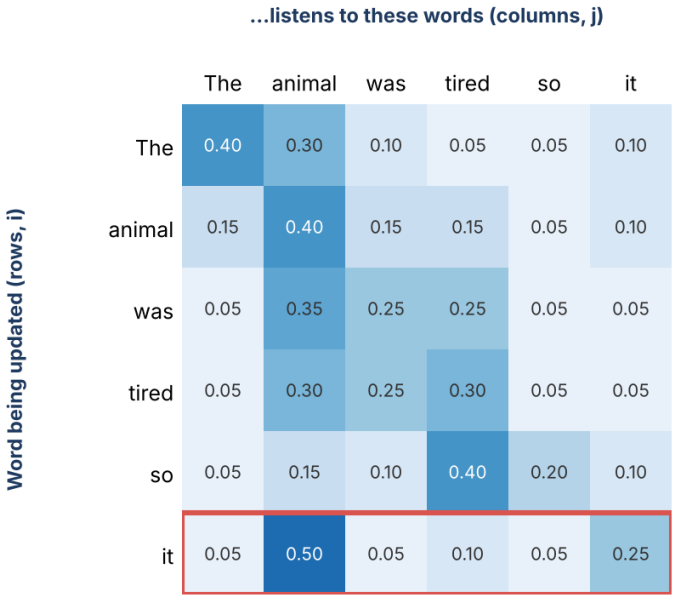

*Same word "it", different attention. A fixed (static) embedding cannot do this; learned attention weights can.*

This is why we need weights α that depend on the sentence: when building the vector for "it", put a high weight on **animal** in sentence A and on **street** in sentence B. A static embedding would give "it"

the same vector in both.

## 2.4 The goal: fill a T × T table

For a sentence with T words we want a table of weights: **row i** = the word being updated, **column j** = the words it listens to. Each row adds up to 1. Once we have the table, each new vector is just "row of weights × the word vectors".




*each row sums to 1 • red row: "it" puts most weight (0.50) on "animal" (illustrative)*

*The T × T attention table (illustrative numbers). The red row shows "it" listening mostly to "animal".*

> **✓ Why this table can be computed in parallel**
>
> In Lecture 1 each row needed the previous row (because of $s_{t-1}$). Now a cell needs only two **input** vectors, e.g. $h_6$ and $h_3$, both already known. No row waits for another row, so **all rows can be computed together**.

## 2.5 Which scoring function should we use?

In the RNN lectures the score was computed like this (general form):

$$e_{tj} = v^T \tanh\left(W_1 s_{t-1} + W_2 h_j\right)$$

Look at what is inside: **three vectors** ($s_{t-1}$, $h_j$ and v), **two linear transformations** ($W_1$, $W_2$), **one non-linearity** (tanh), and a final **dot product** with v. The Transformer paper proposes a simpler design built around the dot product.

**The puzzle:** the old formula had three vectors, but in self-attention we only have two things, $h_i$ and $h_j$. Where do the three vectors come from? **Answer:** create them from each word vector with three learned linear transformations. The three resulting vectors are called the **Query**, **Key** and **Value**, which is the topic of Lecture 3.

**✔ Remember (Lecture 2)**

- Transformer encoder: $x_1 \ldots x_T$ in → $z_1 \ldots z_T$ out, like the RNN encoder but without recurrence.
- Self-attention = every word takes a weighted vote from all words of its own sentence.
- It works for “it” because the weights depend on the sentence. Static embeddings cannot do this.
- No dependence on a previous step means the whole T × T table can be computed in parallel.

**✎ Self-check (answers at the end of the notes)**

**1.** What is the difference between self-attention and cross-attention?

**2.** Why does the scoring function $\alpha_{ij} = f(h_i, h_j)$ allow parallel computation, whereas $f(s_{t-1}, h_i)$ did not?

**3.** In “The animal didn’t cross the street because it was too tired”, which word should “it” attend to most, and what changes if the sentence ends with “…because it was congested”?

LECTURE 3

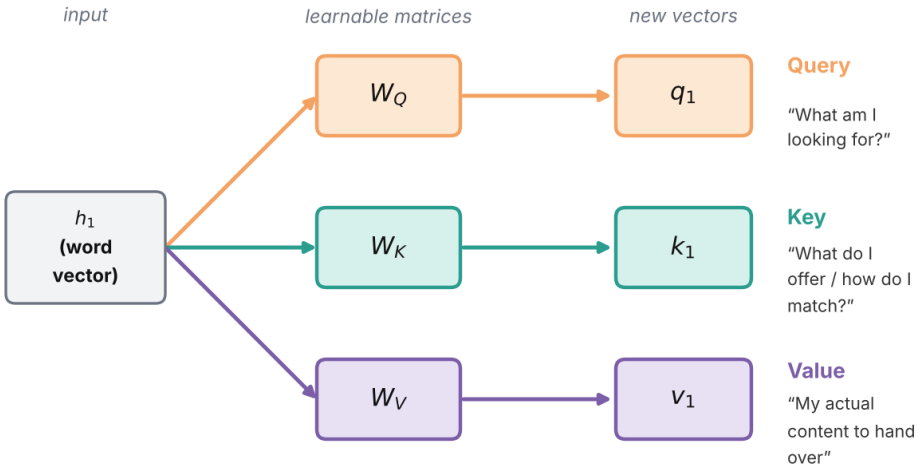
# Self-Attention in Transformers: Contextual Word Embeddings

*Big question: How exactly do we compute self-attention, and why can it run in parallel?*

## 3.1 Step 0: make a query, a key and a value for every word

Every word starts as one vector $h_i$ (say d = 64 numbers). We multiply it by **three different learnable matrices** to get **three new vectors** of the same word:

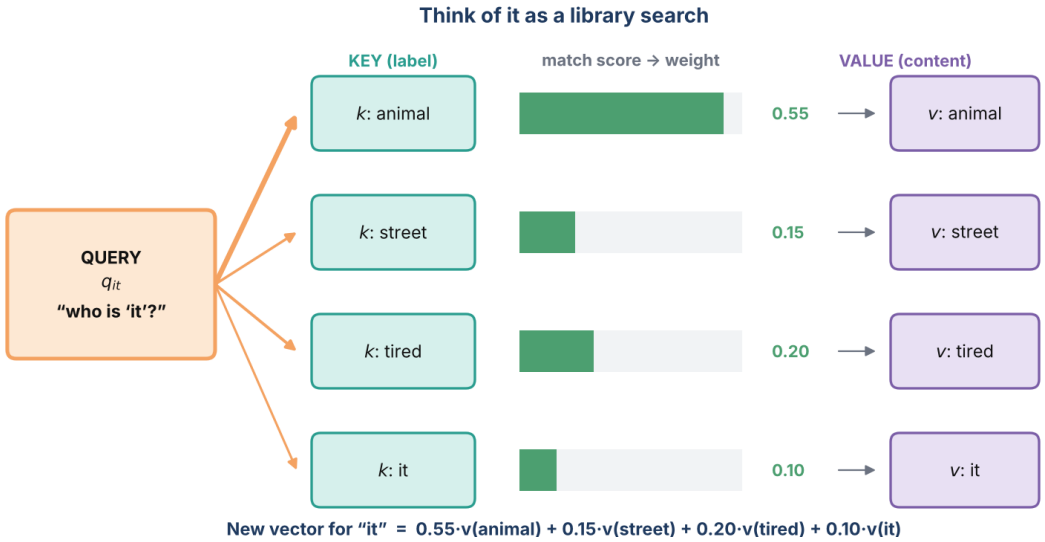
$$q_i = W_Q h_i \qquad k_i = W_K h_i \qquad v_i = W_V h_i$$




*One word vector becomes three vectors through three learned matrices.*

$W_Q$, $W_K$, $W_V$ are **parameters**: they start random and are learned during training. The same three matrices are used for **every** word. If the input has T words, we now have 3 × T vectors.

> ✦ **Mental picture: searching a library**
>
> **Query** = the question you are asking ("who does 'it' refer to?"). **Key** = the label on each book's spine ("what do I contain?"). **Value** = the content inside the book. You compare your query with every key to see which books match, then you read a **mix of the contents**, mostly from the best matches. *(This picture is a study aid and not from the lecture, but it matches how the lecture uses q, k and v.)*




*Query vs. keys gives the weights; the weights then mix the values. (Numbers illustrative.)*

## 3.2 Computing z for one word, step by step

Let us compute the new vector $z_1$ for word 1:

1. **Score** word 1 against every word: $e_{1j} = q_1 \cdot k_j$ (a dot product) for j = 1…T. The query $q_1$ stays **fixed** while the key changes. Word 1 also scores itself ($q_1 \cdot k_1$).
2. **Scale** the scores (divide by √d; more in 3.5).
3. **Softmax** the scores to get weights $\alpha_{1j}$ that are positive and add up to 1.
4. **Mix the values:** $z_1 = \Sigma_j \alpha_{1j} \cdot v_j$.

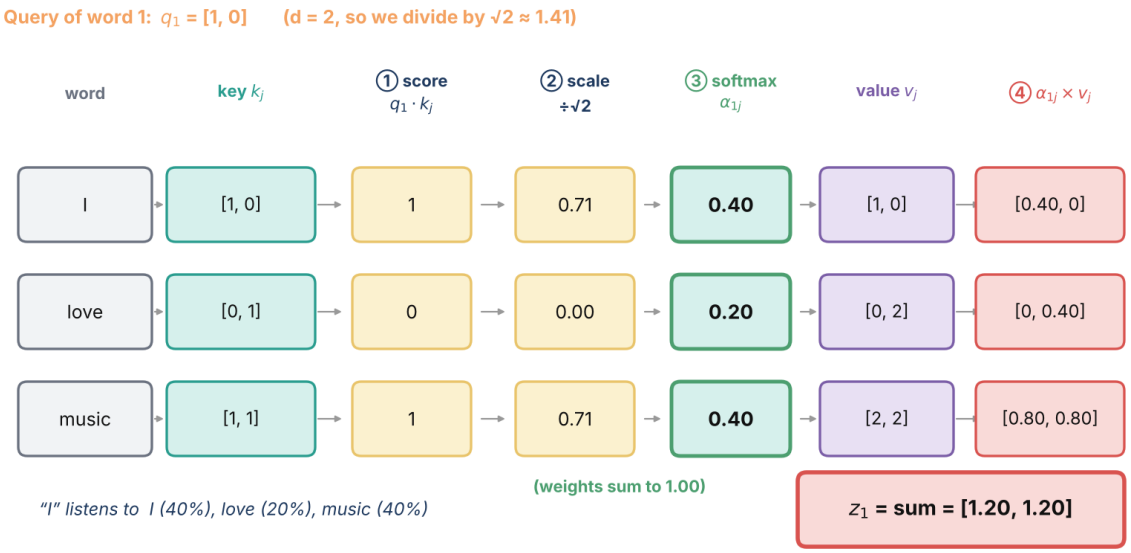
$$e_{ij} = q_i^T k_j \;\Rightarrow\; \alpha_{ij} = \text{softmax}_j(e_{ij}) \;\Rightarrow\; z_i = \sum_{j=1}^{T} \alpha_{ij} v_j$$

For $z_2$ nothing changes except that the query is $q_2$. Same story for every word.

> **✔ Who does what?**
>
> **q and k decide HOW MUCH** each word matters (they produce the weights α). **v decides WHAT** is actually taken from each word (the weighted sum is made of v's). Dot product = similarity: it is large when two vectors point in a similar direction.

## 3.3 A tiny worked example (3 words, d = 2)

Sentence: "I love music". Suppose the projections already gave $q_1$ = [1, 0]; keys k = [1,0], [0,1], [1,1]; values v = [1,0], [0,2], [2,2]. Follow the columns left to right:




*Computing $z_1$. Word "I" gives 40% attention to itself, 20% to "love" and 40% to "music".*

Check by hand: 0.40·[1,0] + 0.20·[0,2] + 0.40·[2,2] = [0.4, 0] + [0, 0.4] + [0.8, 0.8] = **[1.2, 1.2]**. The new vector for "I" is a blend of all three values, mostly from "I" and "music". That is what "contextual" means.

## 3.4 Doing ALL words at once: matrices

Remember why we are here: we do **not** want to compute $z_1$, then $z_2$, then $z_3$ one by one. That would be an RNN again. Luckily everything is a matrix multiplication.

**Step A, all q, k, v together.** Stack the word vectors as columns of a matrix H (size d × T, e.g. 64 × T). One multiplication gives **all** queries, and the same for keys and values:

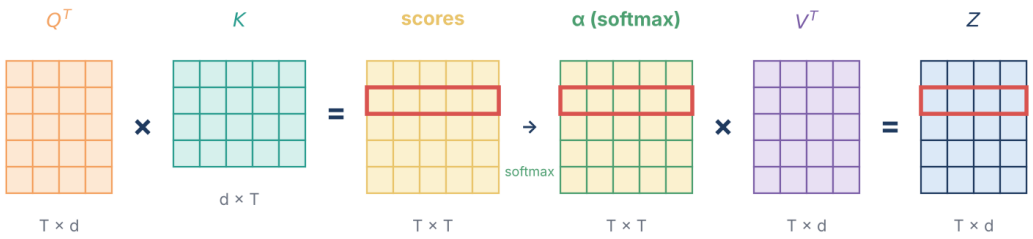
$$Q = W_Q H \qquad K = W_K H \qquad V = W_V H$$

If d = 64, $W_Q$ is 64 × 64 and Q is 64 × T. The key point: **T vectors in → T vectors out**; T never changes. (You may choose a different size $d_1$ for q/k/v; the lecture just uses 64 for simplicity.)

**Step B, all scores together.** $Q^T$ is T × 64 and K is 64 × T, so $Q^T K$ is **T × T**, which is exactly the attention table from Lecture 2. Entry (i, j) = $q_i \cdot k_j$.

**Step C, all outputs together.** Softmax each row, then multiply by $V^T$ (T × 64) to get Z (T × 64): **T new vectors, each 64-dimensional**.

$$Z = \text{softmax}\left(\frac{Q^T K}{\sqrt{d}}\right) V^T$$




*The shapes. One big multiplication produces all T outputs; row 2 of α times V gives $z_2$.*

**✓ Sanity check from the lecture: does row 2 really give $z_2$?**

Row 2 of $Q^TK$ is $[q_2 \cdot k_1, q_2 \cdot k_2, \ldots, q_2 \cdot k_T]$. After softmax these are $\alpha_{21} \ldots \alpha_{2T}$. Multiplying that row by $V^T$ takes a **linear combination of the rows of $V^T$**, giving $\alpha_{21} \cdot v_1 + \alpha_{22} \cdot v_2 + \ldots$ which is exactly $z_2$ from the step-by-step recipe. ✔

**Note on conventions:** the lecture stacks words as *columns*. Many books and papers stack words as *rows* and write the same thing as:

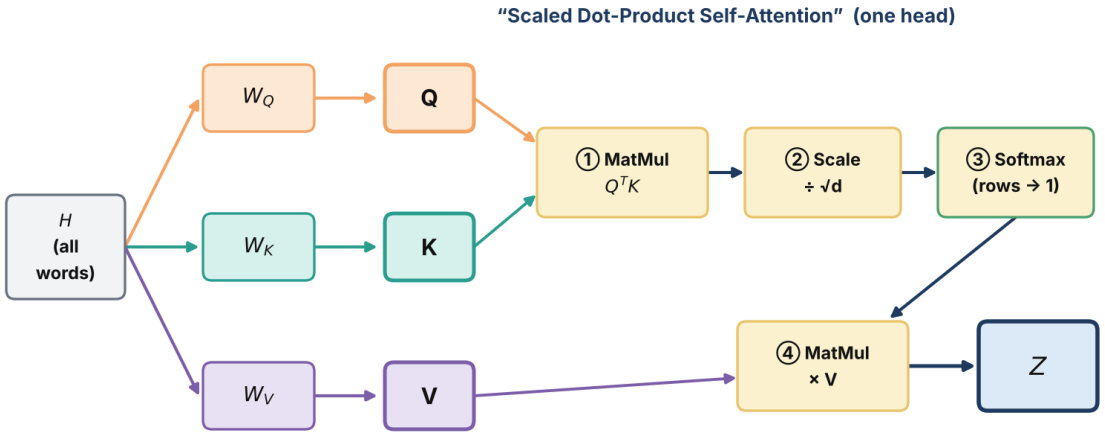
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

## 3.5 "Scaled dot-product" attention: the full block

Because we take dot products and then divide by √d, this block is called **scaled dot-product self-attention**. The lecturer notes there is a justification for the scaling but says that, for practical purposes, you simply divide by a constant related to the dimension.

**◆ Extra intuition (not detailed in the lecture)**

Dot products of long vectors tend to get large. Large numbers make softmax extremely "peaky" (almost all weight on one word). Dividing by √d keeps the numbers in a comfortable range.




*The complete scaled dot-product self-attention block (also called ONE head).*

**Why this beats the RNN:** nothing here depends on a previous time step. The computation is a handful of matrix multiplications, so $z_T$ never has to wait for $z_{T-1}$.

**✔ Remember (Lecture 3)**

- Each word → **q, k, v** via three learned matrices.
- Weights α = softmax(q·k / √d). New vector z = Σ α·v.
- In matrix form: $Z = \text{softmax}(Q^T K/\sqrt{d})\cdot V^T$. T words in, T contextual vectors out, all in parallel.
- The output has the same count of vectors (T) as the input.

**✎ Self-check (answers at the end of the notes)**

**1.** Why do we need three different matrices $W_Q$, $W_K$, $W_V$ instead of using h directly?

**2.** In the worked example, change the weights to [0.1, 0.1, 0.8]. What does $z_1$ become? (v = [1,0], [0,2], [2,2])

**3.** What are the shapes of $Q^T K$ and of the final Z when T = 7 and d = 64?

LECTURE 4

# Multi-Headed Attention in Transformers

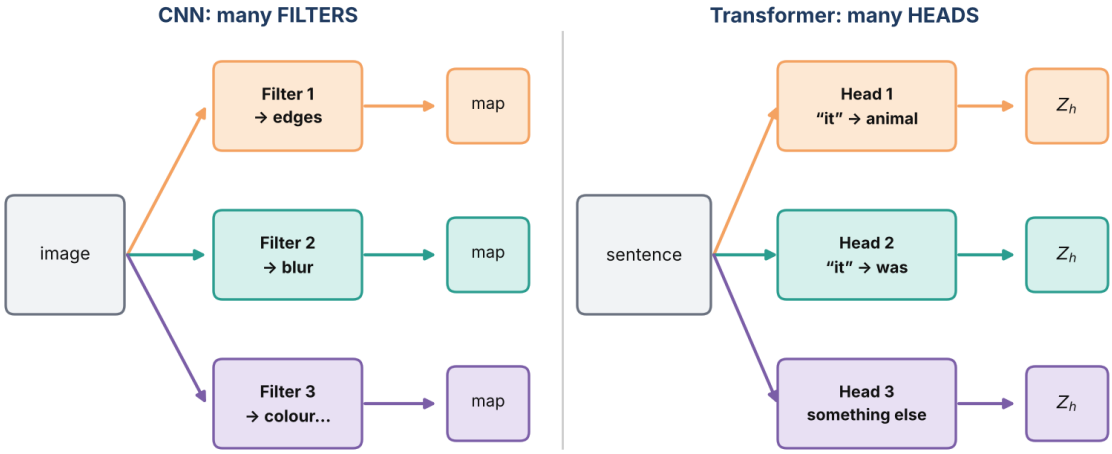
*Big question: One head = one way of looking at the context. Can we have several, and how do we finish an encoder layer?*

## 4.1 Quick recap: one head

Last lecture's block (H → $W_Q$, $W_K$, $W_V$ → Q, K, V → scaled dot-product → Z) is called **one head**. It takes the word vectors $h_1 \ldots h_T$ and returns refined, context-aware vectors $z_1 \ldots z_T$. Now we repeat the same block several times, each copy with **its own** set of W matrices.

## 4.2 Why more than one head?

You have seen this idea before: in a CNN we use **many filters** on the same image, hoping each learns something different (edges, blur, colour…). More filters = richer description. The same logic applies to attention: **one head learns one way of using context, but one way is not enough**.



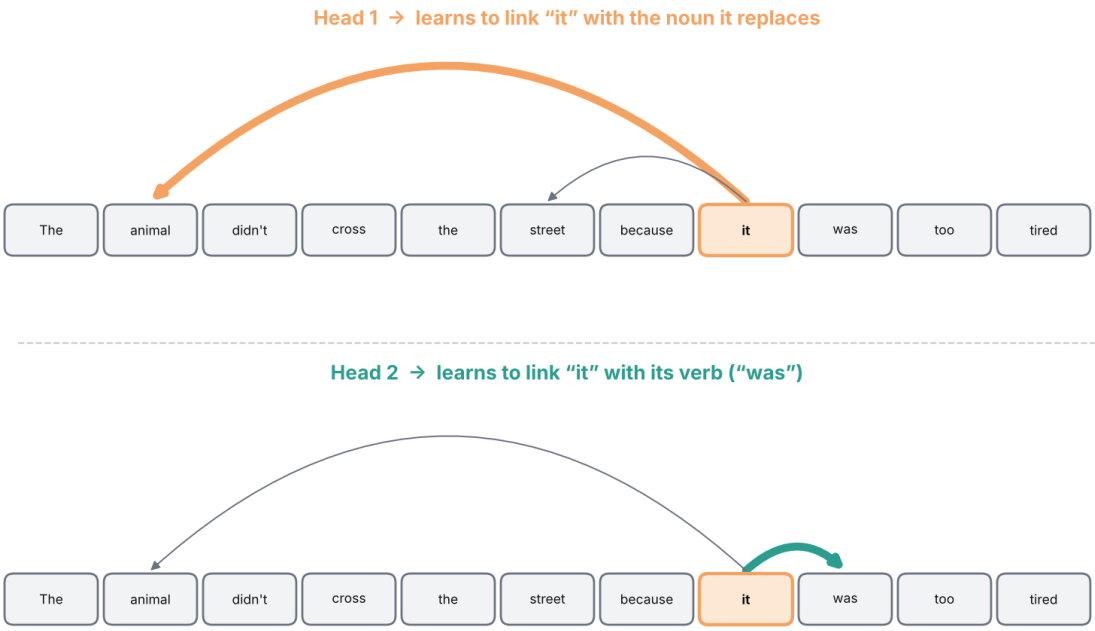

*Each one learns a DIFFERENT useful pattern, and we keep all of them.*

*Many filters in a CNN ↔ many heads in a Transformer. Both give you several different "views" of the same input.*

In "The animal didn't cross the street because it was too tired", to understand "it" we may want to:

- link "it" with **animal** (the noun it stands for), and
- link "it" with **was** (the verb that tells us what "it" is doing, i.e. "it" is its subject).

Both are useful. With one head the weights α must pick one favourite. With two heads, **one head can favour "animal" and the other "was"**.




*Two heads, two different attention patterns for the same word “it”.*

> ⚠ **Important honesty check**
>
> Nobody tells the heads what to look at. There is **no explicit signal** saying “head 1, focus on animal”. We only give the model **more freedom** (more parameters W). If looking at “animal” or “was” **reduces the loss**, training will learn it. In a real trained Transformer the lecturer showed two heads where one gave high attention to “animal” and the other to “was”, so this does happen in practice.

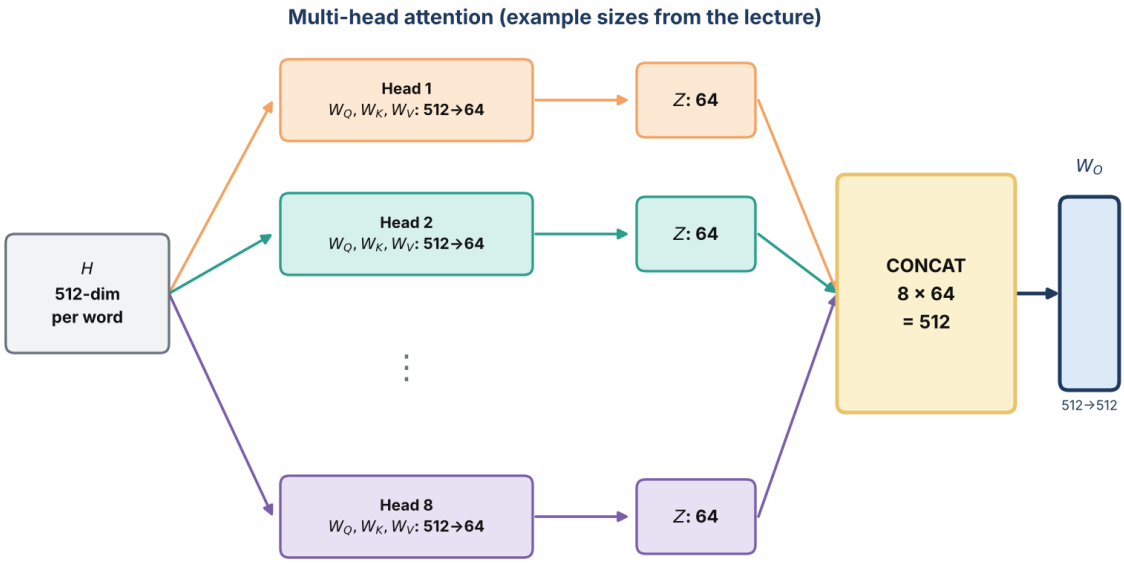
## 4.3 How do we combine the heads? Concatenate, then a linear layer

Each head gives its own vector for every word ($z^{(1)}$, $z^{(2)}$, …). We simply **glue them together side by side (concatenate)** and pass the result through one more learned linear layer ($W_O$):

$$\text{MultiHead} = \text{Concat}(Z_1, Z_2, \ldots, Z_8)\, W_O$$

**Sizes: the part that confuses everyone.** Suppose each word vector is 512-dimensional.

- **Naive idea:** each head outputs 512 numbers. With 8 heads, concatenation gives 8 × 512 = **4096**, which is far too large and needs many more parameters.
- **What is done:** each head **projects down** to 512 / 8 = **64** numbers (W matrices of size 512 → 64). Eight heads × 64 = **512** again.
- Then $W_O$ (e.g. 512 → 512) gives the final output, so the size per word is **unchanged** (512 in, 512 out).




*Multi-head attention with 8 heads of size 64. The concatenation is done per word, so we still have T vectors at the output.*

| Stage | Size per word | Comment |
|---|---|---|
| **Input $h_i$** | 512 | embedding or output of previous layer |
| **q, k, v in each head** | 64 | 512 → 64 projection (one set of W per head) |
| **z from each head** | 64 | 8 heads → eight 64-d vectors |
| **After concatenation** | 8 × 64 = 512 | glue the head outputs side by side |
| **After $W_O$** | 512 | final multi-head output |

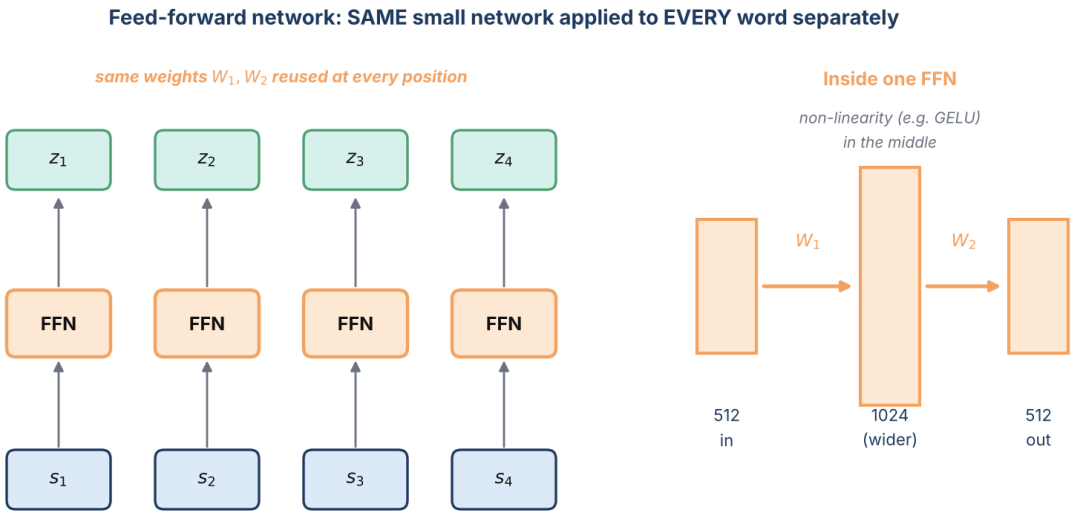
*Lecture example, simplified:* with just 2 heads you could use 256 per head: 2 × 256 = 512 after concatenation. The principle is the same: **choose the per-head size so the concatenation matches the size you want.**

## 4.4 The feed-forward network (FFN)

After attention, every word vector goes through a small **feed-forward neural network**. There is nothing fancy: it acts as a **projection layer** that takes the d-dimensional vector and returns a d-dimensional vector.

- Example sizes: 512 → **1024** (wider, with a non-linearity such as GELU) → 512. *(The original Transformer paper used an even wider middle layer, 2048.)*
- The **same** weights ($W_1$, $W_2$) are used for **every word position**. Each word is processed separately by identical copies of the network.
- Attention is where words **talk to each other**. The FFN is where each word **thinks on its own** about what it just heard.

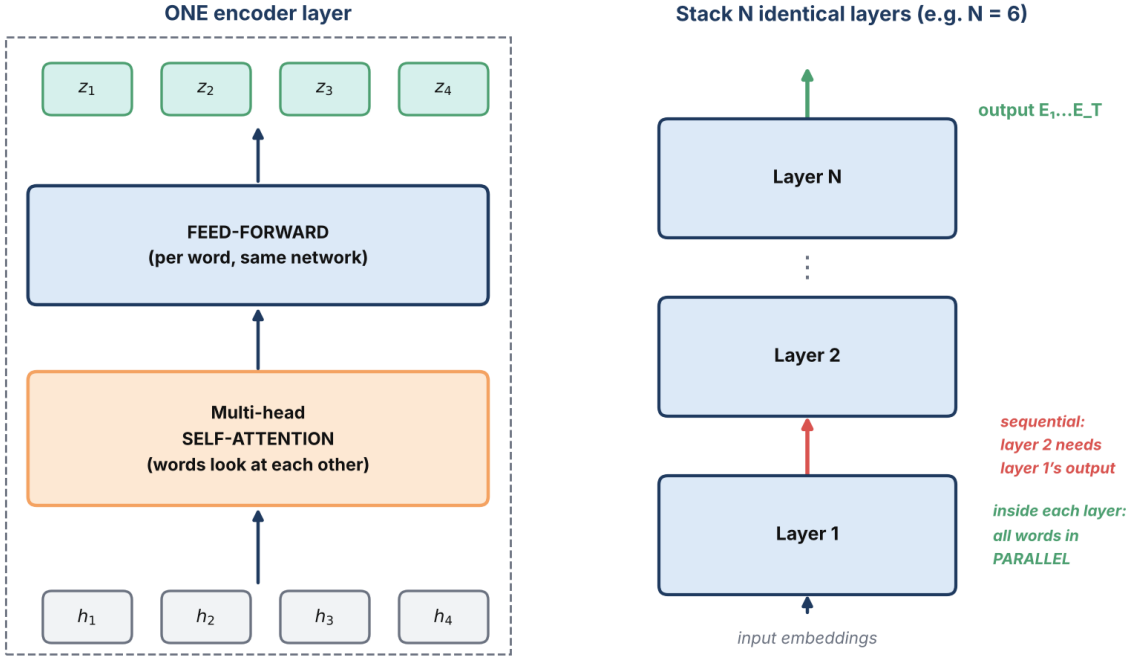
$$\mathrm{FFN}(s_i) = W_2\,\sigma(W_1\,s_i)$$




*The same small network is applied to each word vector independently, so this step is also fully parallel.*

## 4.5 The complete encoder: stack identical layers

One **encoder layer** = **multi-head self-attention** followed by a **feed-forward network**. The Transformer encoder stacks **N identical layers** (typically 6, 8 or 12). The output of one layer becomes the input of the next: T vectors in, T vectors out, every time.




*Left: inside one layer. Right: the stack. Parallel inside a layer, sequential between layers.*

**✓ Where exactly is it parallel?**

**Inside each layer:** all T words, all attention weights and all z's are computed **in parallel** (big matrix multiplications). In an RNN you would first get $z_1$, then $z_2$, then $z_3$…

**Between layers:** still **sequential**. Layer 2 needs layer 1's output first. That is only N steps (e.g. 6), not T steps (the sentence length), which is a huge improvement.

The output of the last layer is called $\mathbf{E_1}$, $\mathbf{E_2}$, …, $\mathbf{E_T}$: one vector per input word that is **contextual** (it knows about the other words) and has passed through **several levels of abstraction** (several layers). In the next lectures the decoder will use these.

## 4.6 RNN encoder vs. Transformer encoder: the final comparison

| | RNN / BiLSTM encoder | Transformer encoder |
|---|---|---|
| **Contextual vectors?** | Yes (through recurrence) | Yes (through self-attention) |
| **How words share information** | pass a hidden state along the sentence | every word looks directly at every other word |
| **Across the T words** | sequential: T steps | parallel: one matrix multiplication |
| **Across the layers** | sequential | sequential (N layers) |
| **Basic building block** | recurrent equation $h_t = f(h_{t-1}, x_t)$ | multi-head self-attention + feed-forward network |
| **Decoder** | generates one word at a time | still generates one word at a time (each output is fed back) |

**✔ Remember (Lecture 4)**

- Head = one scaled dot-product attention block. **Multi-head** = several heads in parallel, each with its own $W_Q$, $W_K$, $W_V$.
- Heads give **different views** of the context (like CNN filters); the model learns what each should do by minimising the loss.
- Combine: **concatenate** the heads, then one linear layer. Typical: 8 heads × 64 = 512.
- FFN = same small network on each word (512 → 1024 → 512). Encoder layer = multi-head attention + FFN. Encoder = N such layers → $E_1 \ldots E_T$.

**✎ Self-check (answers at the end of the notes)**

**1.** Why not simply make each head output 512 numbers? What goes wrong with 8 heads?

**2.** Where is the Transformer encoder parallel, and where is it still sequential?

**3.** Name two differences between what the attention sub-layer does and what the feed-forward sub-layer does.

REVISION

# One-page cheat sheet

*Big question: Everything from Week 1 on a single page.*

## The whole encoder in 9 steps

1. **Embed** the T words: H (each word is a d-dimensional vector, e.g. d = 512).
2. **Project** with learned matrices per head: Q = $W_Q$H, K = $W_K$H, V = $W_V$H.
3. **Score** every pair of words: $Q^T$K (a T × T table).
4. **Scale** by 1/√d.
5. **Softmax** each row → weights α (each row adds up to 1).
6. **Mix the values:** Z = α·$V^T$ → one new contextual vector per word.
7. **Multi-head:** do steps 2–6 for 8 heads, **concatenate** the results, multiply by $W_O$.
8. **Feed-forward** on each word (same network at every position).
9. **Repeat** steps 2–8 for N layers (6/8/12). The final output is $E_1$…$E_T$.

## Formulas

$$q_i = W_Q\, h_i \qquad k_i = W_K\, h_i \qquad v_i = W_V\, h_i$$

$$e_{ij} = q_i^T k_j \;\Rightarrow\; \alpha_{ij} = \text{softmax}_j(e_{ij}) \;\Rightarrow\; z_i = \sum_{j=1}^{T} \alpha_{ij}\, v_j$$

$$Z = \text{softmax}\left(\frac{Q^T K}{\sqrt{d}}\right) V^T$$

$$\text{MultiHead} = \text{Concat}(Z_1, Z_2, \ldots, Z_8)\; W_O$$

$$\text{FFN}(s_i) = W_2\; \sigma(W_1\, s_i)$$

## Why the Transformer beat the RNN

- No recurrence → nothing waits for the previous word → all T words processed in parallel.
- Every word sees every other word **directly** (not through a long chain of hidden states).
- Still gives **contextual** representations, so we lose nothing we liked about RNNs.

## Common confusions, cleared up

**“Are q, k, v three different words?”**

No. They are three different **views of the same word vector**, made by three different learned matrices. Each word has one q, one k and one v.

**“Does a word attend to itself?”**

Yes. $q_1 \cdot k_1$ is computed like any other pair. (A word’s own meaning is usually important for its own representation.)

**“Who tells the head to look at ‘animal’?”**

Nobody. The only signal is the loss. If putting weight on “animal” helps reduce the loss, training learns to do that.

**“Is the whole Transformer parallel?”**

The **encoder** is parallel across words within a layer. Layers are still sequential, and the **decoder** still writes one output word at a time (each output is fed back as input).

**“Why do the letters h, s, z keep changing?”**

Because the lecturer re-uses symbols while transitioning from RNNs. See the decoder-ring table at the start. Track the idea: *T vectors in, T better vectors out*.

**“If every head sees the same input, why aren’t they identical?”**

Each head has its **own** $W_Q$, $W_K$, $W_V$ (different random starting values), so training can push them toward different patterns.

**“What is the difference between attention weight α and the value v?”**

α is **how much** (a single number per pair of words). v is **what** (a vector of content). The output is $\Sigma\ \alpha \cdot v$.

## Answers to the self-check questions

### Lecture 1

1. Because $h_2 = f(h_1, x_2)$ literally needs $h_1$ as an input. The context is passed along through this chain, so each step waits for the previous one.
2. A plain embedding gives a word one fixed vector everywhere. A contextual representation changes with the sentence (e.g., “bank” in “river bank” vs. “bank account”).
3. Within a time step, $\alpha_{ti}$ needs only $s_{t-1}$ and $h_i$, both known. Across time steps, $\alpha_t$ needs $s_{t-1}$, which needs $c_{t-1}$, which needs $\alpha_{t-1}$: a chain.

### Lecture 2

1. Cross-attention is between the decoder and the encoder’s words. Self-attention is among the words of the same sentence (inside the encoder, or inside the decoder).
2. Because $f(h_i, h_j)$ uses only the input vectors, which we all have at the start, whereas $f(s_{t-1}, h_i)$ needed the decoder state from the previous step.
3. “It” attends most to **animal** in the first sentence. With “…because it was congested” it should attend most to **street**.

### Lecture 3

1. They give each word three different roles: a query (what I look for), a key (how I can be matched) and a value (what I hand over). Using h for all three would make the model far less flexible. The matrices are also what the model **learns**.
2. $z_1 = 0.1 \cdot [1,0] + 0.1 \cdot [0,2] + 0.8 \cdot [2,2] = [0.1, 0] + [0, 0.2] + [1.6, 1.6] =$ **[1.7, 1.8]**.
3. $Q^T K$ is 7 × 7 (T × T). Z is 7 × 64 (T × d): seven vectors of 64 numbers.

### Lecture 4

1. Concatenating 8 vectors of 512 gives 4096 numbers per word: the following linear layer would need many more parameters. So each head projects down to 512/8 = 64, and the 8 heads together give 512 again.
2. Parallel: across all T words inside every layer. Sequential: from layer to layer (layer 2 needs layer 1’s output). Also the decoder generates words one by one.
3. Attention mixes information **between** words (every word looks at all words). The FFN works on **each word separately** with the same network, projecting its vector through a wider hidden layer and a non-linearity.

> **→ What comes next**
>
> Next we meet the **decoder side** of the Transformer: its self-attention, the **encoder–decoder (cross) attention** that connects the decoder to the E vectors you built this week, and its feed-forward layers.